

Student

The model is designed. Now how do I actually build it - the SQL to implement SCD Type 2, generate surrogate keys, handle late data, correct mistakes, and validate quality on every load?

Claude • AI Assistant

Design without implementation is architecture fiction. This chapter covers the ETL mechanics that turn a dimensional model on paper into a warehouse that loads reliably every day. Five precision subsections, all at the SQL level:

1. **SCD Type 2 with SQL MERGE** - detecting changes, expiring current rows, inserting new versions.
2. **Surrogate key generation** - sequences, identity columns, and the natural-key-to-surrogate mapping pipeline.
3. **Late-arriving dimensions** - when a fact arrives before its dimension row exists.
4. **Slowly changing facts** - handling corrections and restatements to already-loaded measures.
5. **Data quality rules** - the validation layer that runs before every fact load.

CONCEPT CARD

The ETL load sequence for a dimensional warehouse:

1. **Extract** - pull delta records from source systems (changed since last load).
2. **Validate** - run data quality checks on staging data before touching the warehouse.
3. **Load dimensions first** - apply SCD logic; generate surrogate keys for new/changed rows.
4. **Look up surrogate keys** - resolve natural keys from source to warehouse surrogate keys.
5. **Load facts** - insert resolved fact rows with foreign key integers pointing to dimension tables.
6. **Post-load validation** - verify row counts, referential integrity, aggregate checks.

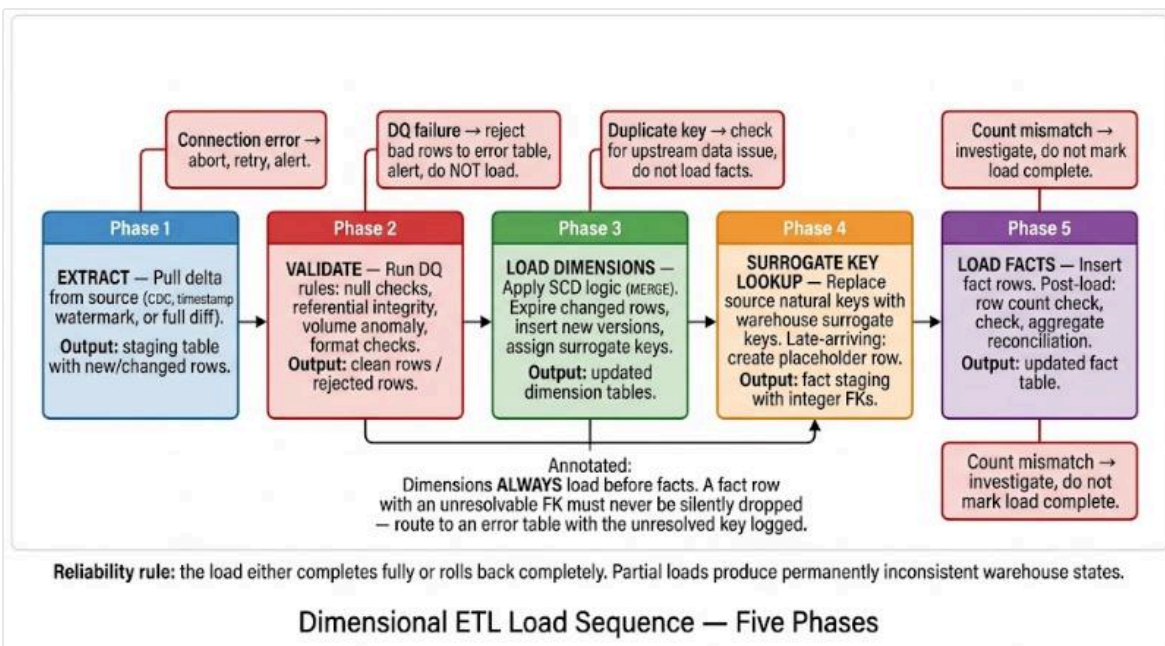
SUMMARY CARD

By the end of this chapter you will be able to:

- Write a complete SCD Type 2 MERGE statement that handles inserts, updates (expire + insert), and no-changes correctly.
- Design a surrogate key pipeline that decouples the warehouse from source system key changes.
- Handle late-arriving dimension records without losing or corrupting fact data.
- Implement fact correction patterns (negating entry, in-place update with audit) correctly.
- Build a five-rule data quality validation layer that gates every dimension and fact load.

🔮 AI DEEP DIVE PROMPT

Warm-up: "Explain the complete ETL sequence for loading a Type 2 slowly changing dimension. What are the three possible outcomes for each incoming source row (new record, changed record, unchanged record), and what SQL action does each outcome trigger?"



Dimensional ETL Load Sequence - Five Phases

SCD Type 2 MERGE - Three Outcomes and Their SQL Actions

Source row vs current dim row	Outcome	SQL action in MERGE
Natural key not in dimension	New record - never seen before	INSERT new row: surrogate key generated, rowstart = today, rowend = 9999-12-31, is_current = 1

Source row vs current dim row	Outcome	SQL action in MERGE
Natural key exists AND one or more tracked attributes changed	Changed record - requires new version	UPDATE current row: set <i>rowend</i> = <i>today</i> - 1, <i>iscurrent</i> = 0. Then INSERT new row with updated attrs, <i>rowstart</i> = <i>today</i> , <i>iscurrent</i> = 1
Natural key exists AND all tracked attributes unchanged	Unchanged record - no action needed	No SQL operation (MERGE WHEN MATCHED AND no-change condition → do nothing)
Natural key exists AND <i>row_end</i> < today (historical)	Historical row - skip entirely	No SQL operation (filter WHERE <i>is_current</i> = 1 before MERGE)

sql – Complete ETL skeleton – dimension load then fact load with surrogate key lookup

```
-- =====
-- PHASE 1: Stage source delta into a staging table
-- =====
INSERT INTO stg_customer
SELECT * FROM source_crm.customers
WHERE updated_at > (SELECT MAX(last_load_ts) FROM etl_watermarks
                    WHERE table_name = 'dim_customer');

-- =====
-- PHASE 2: Data quality gate (abort load if failures found)
-- =====
DO $$
DECLARE v_bad INT;
BEGIN
    SELECT COUNT(*) INTO v_bad FROM stg_customer
    WHERE customer_id IS NULL OR customer_name IS NULL
        OR email NOT LIKE '%@%';
    IF v_bad > 0 THEN
        RAISE EXCEPTION 'DQ FAIL: % rows failed null/format checks', v_bad;
    END IF;
END $$;

-- =====
-- PHASE 3: SCD Type 2 dimension load (MERGE – see Section 4-1)
-- =====
-- (Full MERGE statement in section 4-1 code block)
```

```
-- =====
-- PHASE 4: Surrogate key lookup for fact staging
-- =====

INSERT INTO stg_sales_resolved
SELECT
    d.date_key,
    c.customer_key,
    p.product_key,
    s.store_key,
    src.transaction_number,
    src.quantity,
    src.gross_revenue,
    src.discount_amount
FROM    stg_sales_source src
JOIN    dim_date          d ON d.calendar_date = src.sale_date
JOIN    dim_customer      c ON c.customer_id   = src.customer_id
                        AND c.is_current      = 1
JOIN    dim_product       p ON p.product_code  = src.product_code
                        AND p.is_current      = 1
JOIN    dim_store         s ON s.store_code    = src.store_code;

-- =====
-- PHASE 5: Load fact table
-- =====

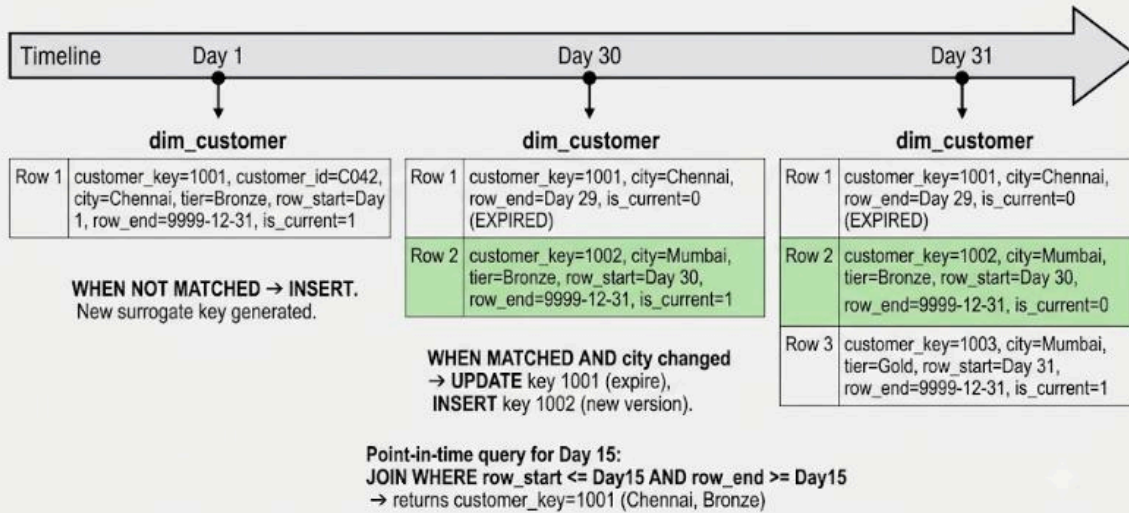
INSERT INTO fact_sales_transaction
SELECT date_key, customer_key, product_key, store_key,
       transaction_number, quantity, gross_revenue, discount_amount,
       gross_revenue - discount_amount AS net_revenue
FROM    stg_sales_resolved;
```

SCD TYPE 2 WITH SQL MERGE — THE FULL IMPLEMENTATION



Claude • AI Assistant

SCD Type 2 Row Lifecycle — Three Loads, Two Versions



Fact rows loaded on Day 1 forever carry customer_key=1001. Fact rows after Day 30 carry 1002. Historical accuracy is encoded in the FK itself.

SCD Type 2 Row Lifecycle - Three Loads, Two Versions

sql – Complete SCD Type 2 MERGE – two-step expire-then-insert pattern

```
-- =====
-- DIM_CUSTOMER: SCD Type 2 with all required columns
-- =====

CREATE TABLE dim_customer (
    customer_key    INT                PRIMARY KEY, -- surrogate (sequence)
    customer_id     VARCHAR(20)        NOT NULL,   -- natural key from CRM
    customer_name   VARCHAR(200)       NOT NULL,   -- TRACKED
    email           VARCHAR(200),       -- TRACKED
    city            VARCHAR(100),       -- TRACKED
    loyalty_tier    VARCHAR(15),       -- TRACKED
    acquisition_ch  VARCHAR(50),       -- NOT tracked (Type 1
    row_start_date  DATE                NOT NULL,
    row_end_date    DATE                NOT NULL DEFAULT '9999-12-31',
    is_current      TINYINT             NOT NULL DEFAULT 1
);

-- =====
-- STEP 1: MERGE – expire changed rows + flag new/changed
-- Creates a temp table of rows needing new version insertion
-- =====

CREATE TEMP TABLE stg_to_insert AS
SELECT src.*
FROM   stg_customer src -- staging table from source system
WHERE  1 = 0;          -- empty shell with same columns
```

```

MERGE INTO dim_customer tgt
USING (
    SELECT customer_id, customer_name, email, city, loyalty_tier
    FROM   stg_customer
) src
ON (tgt.customer_id = src.customer_id AND tgt.is_current = 1)

-- Case 1: NEW record – never seen before
WHEN NOT MATCHED THEN
    INSERT (customer_key, customer_id, customer_name, email,
            city, loyalty_tier, row_start_date, row_end_date, is_current)
    VALUES (NEXT VALUE FOR seq_customer_key,
            src.customer_id, src.customer_name, src.email,
            src.city, src.loyalty_tier,
            CURRENT_DATE, '9999-12-31', 1)

-- Case 2: CHANGED record – expire the current row
WHEN MATCHED
    AND (tgt.customer_name <> src.customer_name
        OR tgt.email <> src.email
        OR tgt.city <> src.city
        OR tgt.loyalty_tier <> src.loyalty_tier)
    THEN
        UPDATE SET
            tgt.row_end_date = CURRENT_DATE - INTERVAL '1 day',
            tgt.is_current = 0

-- Case 3: UNCHANGED record – no action (implicit in MERGE)
;

-- =====
-- STEP 2: INSERT new versions for all changed rows
-- =====

INSERT INTO dim_customer
    (customer_key, customer_id, customer_name, email,
     city, loyalty_tier, row_start_date, row_end_date, is_current)
SELECT
    NEXT VALUE FOR seq_customer_key,
    src.customer_id, src.customer_name, src.email,
    src.city, src.loyalty_tier,
    CURRENT_DATE, '9999-12-31', 1
FROM   stg_customer      src
JOIN   dim_customer      tgt
    ON  tgt.customer_id = src.customer_id
    AND tgt.is_current = 0 -- just expired in Step 1
    AND tgt.row_end_date = CURRENT_DATE - INTERVAL '1 day';

```

```
-- =====
-- CONSISTENCY CHECK: detect is_current / row_end_date mismatch
-- =====
SELECT customer_id, customer_key, row_end_date, is_current,
       'MISMATCH' AS issue
FROM   dim_customer
WHERE  (is_current = 1 AND row_end_date <> '9999-12-31')
       OR (is_current = 0 AND row_end_date = '9999-12-31');
-- Expected: 0 rows
```

Student

I know what SCD Type 2 is conceptually. Now show me the actual SQL that implements it.

Claude • AI Assistant

SCD Type 2 is conceptually simple - expire the old row, insert a new one - but the SQL must handle three distinct cases correctly and atomically: new records that have never been seen, changed records that need a new version, and unchanged records that need no action. The SQL MERGE statement handles all three in one pass.

CONCEPT CARD

SCD Type 2 dimension table - required columns:

1. **Natural key** - the source system identifier (`customer_id VARCHAR`). Used for lookup; not the primary key.
2. **Surrogate key** - the warehouse primary key (`customer_key INT` generated by sequence). Every fact table FK points here.
3. **Tracked attributes** - the attributes that trigger a new version when they change (`customer_name` , `email` , `address` , `loyalty_tier`).
4. **`row\_start\_date DATE`** - the date this version became effective.
5. **`row\_end\_date DATE`** - the date this version expired. Convention: `9999-12-31` means 'currently active.'
6. **`is\_current TINYINT`** - 1 for the active row, 0 for historical. Redundant with `rowenddate` but dramatically speeds up current-row lookups in ETL.

CONCEPT CARD

The three MERGE outcomes in detail:

1. **WHEN NOT MATCHED (new record)** - the source natural key has no matching row in the dimension. Insert a new row: generate surrogate key, set `row_start = CURRENT_DATE` , `row_end = '9999-12-31'` , `is_current = 1` .
2. **WHEN MATCHED AND changed** - the natural key exists AND at least one tracked attribute differs.

Two operations: (a) UPDATE the current row to expire it: `row_end = CURRENT_DATE - 1` , `is_current = 0` . (b) INSERT the new version with updated attributes.

Note: most SQL MERGE statements cannot INSERT and UPDATE in the same WHEN MATCHED clause - this requires a workaround (see code block).

3. **WHEN MATCHED AND unchanged** - the natural key exists AND all tracked attributes are identical. No operation.

CONCEPT CARD

The MERGE workaround for expire + insert:

1. Most databases (SQL Server, Snowflake, BigQuery) cannot INSERT in a WHEN MATCHED clause.
2. **Two-step approach** - Step 1: MERGE to expire changed rows (UPDATE only). Step 2: INSERT new versions for all source rows where the natural key was found as changed.
3. **Staging helper** - write changed source rows to a `stg_changed` staging table during the MERGE, then INSERT from `stg_changed` in a separate statement.
4. **Snowflake MERGE** - supports WHEN MATCHED THEN UPDATE and WHEN NOT MATCHED THEN INSERT only. The expire-and-reinsert requires the two-step pattern.
5. **PostgreSQL / MySQL** - use INSERT ... ON CONFLICT with triggers or explicit multi-step logic; native MERGE syntax varies by version.

EXAMPLE CARD

Step-by-step walkthrough for customer 'Ravi Sharma':

1. **Load 1 (Jan 1)** - source: `{customer_id: 'C001', name: 'Ravi Sharma', city: 'Hyderabad', tier: 'Silver'}` . NOT MATCHED → INSERT: `customer_key=1001`, `row_start=2024-01-01`, `row_end=9999-12-31`, `is_current=1` .
2. **Load 2 (Mar 15)** - source: `{customer_id: 'C001', name: 'Ravi Sharma', city: 'Bengaluru', tier: 'Silver'}` . MATCHED AND `city` changed → UPDATE key 1001: `row_end=2024-03-14`, `is_current=0` . INSERT: `customer_key=1002`, `city='Bengaluru'`, `row_start=2024-03-15`, `is_current=1` .
3. **Load 3 (Mar 16)** - source: same as Load 2. MATCHED AND unchanged → no action.
4. **Point-in-time query** - fact rows loaded on Jan 15 have `customer_key=1001` (Hyderabad). Fact rows loaded after Mar 15 have `customer_key=1002` (Bengaluru). Historical accuracy is preserved in the fact table FKs.

🔗 ANALOGY

The SCD Type 2 MERGE is like **a hospital's patient record system**. Each time a patient's address changes, the old record is marked as superseded (expire = set end date) and a new record is created with the current address.

A medical record from 2020 still says 'Patient lived in Hyderabad at time of treatment' - historical accuracy is preserved. The surrogate patient ID links all versions.

⚠️ WARNING

`is\_current` and `row\_end\_date` can get out of sync if the ETL fails mid-run. After a failed load, always run a consistency check: every dimension row where `row_end_date = '9999-12-31'` should have `is_current = 1`, and every row where `is_current = 1` should have `row_end_date = '9999-12-31'`.

Any mismatch indicates a partial-write failure requiring manual correction.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Using `is_current = 1` as the sole current-row indicator without `row_start_date` and `row_end_date`. You can tell which row is current NOW, but you cannot answer 'which version of this customer was active on July 4th last year?' - point-in-time queries are impossible.

Correct Way: Always maintain all three: `row_start_date`, `row_end_date`, and `is_current`. The dates enable point-in-time queries; the flag enables fast ETL lookups. Both are needed.

EXAM TIP CARD

Interview classic: "Walk me through the SQL logic for a Type 2 SCD load." Winning answer:
*"Two steps. Step 1: MERGE staging into dim on natural key.

WHEN NOT MATCHED: INSERT new row (surrogate generated, rowstart=today, rowend=9999-12-31, iscurrent=1). WHEN MATCHED AND any tracked attribute changed: UPDATE current row setting rowend=today-1, is_current=0.

Step 2: INSERT new versions for all rows identified as changed in Step 1. Three columns drive everything: rowstartdate, rowenddate, is_current."*

🔥 EXERCISE

Load these three source customer rows into a Type 2 dimension in sequence: (a) Day 1:

`{id:'C042', name:'Priya Nair', city:'Chennai', tier:'Bronze'}`. (b) Day 30:

`{id:'C042', name:'Priya Nair', city:'Mumbai', tier:'Bronze'}`.

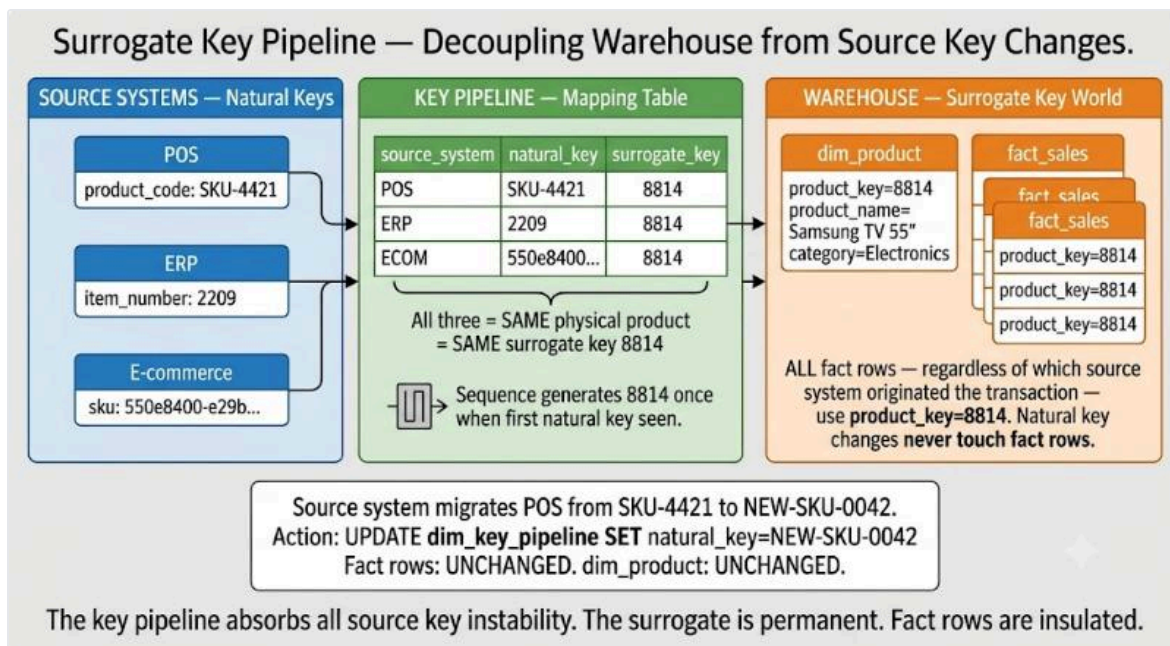
(c) Day 31: `{id:'C042', name:'Priya Nair', city:'Mumbai', tier:'Gold'}`. Write out the full dimension table state after each load, including all surrogate keys, dates, and `is_current` flags.

AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Write a complete SQL Server / Snowflake SCD Type 2 implementation for a product dimension with tracked attributes: product_name, category, unit_cost, is_discontinued. Include: the dimension DDL with all required columns, a two-step MERGE pattern (expire step + insert step), a point-in-time query that retrieves the product version active on a specific historical date, and a consistency check query that detects is_current / row_end_date mismatches."

SURROGATE KEY GENERATION — SEQUENCES, IDENTITY, AND THE KEY PIPELINE

Claude • AI Assistant



Surrogate Key Pipeline - Decoupling Warehouse from Source Key Changes

sql — Surrogate key pipeline DDL and ETL lookup logic

```
-- =====
-- KEY PIPELINE TABLE: natural key → surrogate key mapping
-- =====

CREATE TABLE dim_key_pipeline (
    pipeline_id      SERIAL          PRIMARY KEY,
    source_system    VARCHAR(50)    NOT NULL,    -- 'POS', 'ERP', 'ECOM'
    entity_type      VARCHAR(50)    NOT NULL,    -- 'product', 'customer'
    natural_key      VARCHAR(200)   NOT NULL,    -- source system identifier
```

```

surrogate_key INT NOT NULL, -- FK → the dimension table
first_seen DATE NOT NULL DEFAULT CURRENT_DATE,
last_seen DATE NOT NULL DEFAULT CURRENT_DATE,
UNIQUE (source_system, entity_type, natural_key)
);

-- =====
-- SURROGATE KEY SEQUENCE (one per dimension)
-- =====
CREATE SEQUENCE seq_product_key START 1 INCREMENT 1;
CREATE SEQUENCE seq_customer_key START 1 INCREMENT 1;

-- =====
-- ETL LOOKUP FUNCTION: return surrogate, generate if new
-- =====
CREATE OR REPLACE FUNCTION get_or_create_product_key(
    p_source_system VARCHAR,
    p_natural_key VARCHAR,
    p_product_code VARCHAR,
    p_product_name VARCHAR,
    p_category_name VARCHAR
) RETURNS INT AS $$
DECLARE
    v_surrogate INT;
BEGIN
    -- Try to find existing mapping
    SELECT surrogate_key INTO v_surrogate
    FROM dim_key_pipeline
    WHERE source_system = p_source_system
        AND entity_type = 'product'
        AND natural_key = p_natural_key;

    IF v_surrogate IS NULL THEN
        -- New natural key: generate surrogate, add to pipeline, insert dim
row
        v_surrogate := NEXTVAL('seq_product_key');

        INSERT INTO dim_key_pipeline
            (source_system, entity_type, natural_key, surrogate_key)
        VALUES (p_source_system, 'product', p_natural_key, v_surrogate);

        INSERT INTO dim_product
            (product_key, product_code, product_name, category_name,
             row_start_date, row_end_date, is_current)
        VALUES (v_surrogate, p_product_code, p_product_name, p_category_name,
            CURRENT_DATE, '9999-12-31', 1);
    
```

```

ELSE
    -- Known key: update last_seen
    UPDATE dim_key_pipeline SET last_seen = CURRENT_DATE
    WHERE source_system = p_source_system
        AND entity_type = 'product'
        AND natural_key = p_natural_key;
END IF;

RETURN v_surrogate;
END;
$$ LANGUAGE plpgsql;

-- =====
-- DETECT ORPHANED SURROGATE KEYS
-- =====
SELECT f.product_key, 'Surrogate in fact but not in dim_product' AS issue
FROM fact_sales_transaction f
LEFT JOIN dim_product p ON f.product_key = p.product_key
WHERE p.product_key IS NULL;
-- Expected: 0 rows. Any rows = referential integrity failure.

```

Student

How do surrogate keys get generated, and how do I maintain the mapping between source natural keys and warehouse surrogate keys?

Claude • AI Assistant

Surrogate key generation is not just a technical detail - it is the mechanism that **decouples the warehouse from source system instability**. A source system can renumber its customers, merge two product catalogs, or be replaced entirely.

The warehouse surrogate keys remain stable, and the natural-key-to-surrogate mapping absorbs the change without touching a single fact row.

CONCEPT CARD

Three surrogate key generation mechanisms:

1. **Database sequence / identity column** - **SERIAL** (PostgreSQL), **IDENTITY** (SQL Server, Snowflake), **AUTO_INCREMENT** (MySQL). The database generates the next integer automatically on INSERT. Simple, reliable, the standard approach for most warehouses.

2. **Key pipeline table** - a dedicated `dim_key_mapping(natural_key, surrogate_key, source_system, table_name)` table maintained by the ETL. Enables cross-source key reconciliation and supports natural key changes.
3. **Hash surrogate** - `MD5(source_system || natural_key)` → a 128-bit hash stored as a BIGINT or CHAR(32). Enables key generation without a centralized sequence. Used in distributed pipelines (dbt, Spark). Risk: hash collision (astronomically rare but non-zero).

CONCEPT CARD

Why surrogate keys decouple the warehouse from source systems:

1. **Source key instability** - source systems renumber primary keys during migrations, mergers, or re-platforming. Every time this happens, a natural-key warehouse must update millions of fact rows. A surrogate-key warehouse updates only the mapping table.
2. **Multi-source integration** - the Sales system calls a product 'SKU-4421'. The ERP calls it 'ITEM-2209'. The warehouse assigns surrogate key 8814 to the same physical product. Both source keys map to 8814 via the key pipeline.
3. **Surrogate key stability** - once assigned, a surrogate key NEVER changes, even if every attribute of the dimension row changes via Type 2. The surrogate key is the warehouse's permanent identity for that entity.
4. **Fact table insulation** - a fact row loaded in 2019 with `customer_key = 5503` still points to the correct customer in 2024, even if the customer's natural key, name, address, and tier have all changed. No fact rows ever need updating due to dimension changes.

CONCEPT CARD

The key pipeline - maintaining the mapping:

1. **Purpose** - the key pipeline translates incoming source natural keys to warehouse surrogate keys during the surrogate key lookup phase of ETL.
2. **Structure** - `dim_key_pipeline(source_system VARCHAR, entity_type VARCHAR, natural_key VARCHAR, surrogate_key INT, first_seen DATE, last_seen DATE)`.
3. **Lookup logic** - for each source row, JOIN to the pipeline table. If found: use existing surrogate. If not found: generate new surrogate, INSERT into pipeline, then INSERT into dimension.
4. **Type 2 and the pipeline** - each Type 2 version of a dimension row gets a DIFFERENT surrogate key. The pipeline maps `natural_key → [1001, 1002, 1003]` (one per version). The ETL uses the `is_current = 1` version for new fact loads.

EXAMPLE CARD

Natural key change scenario:

1. Source system migrates from numeric to UUID-based customer IDs. Old: `customer_id = 'C001'`. New: `customer_id = '550e8400-e29b-41d4-a716-446655440000'`.

2. Without surrogate keys: every fact row containing the old customer FK must be updated. Potentially billions of rows.
3. With surrogate keys: update the key pipeline table - `natural_key` changes from `'C001'` to the UUID. The `surrogate_key` remains `customer_key = 1001`. No fact rows touched.
4. The key pipeline row: `UPDATE dim_key_pipeline SET natural_key = '550e8400-...' WHERE source_system='CRM' AND entity_type='customer' AND natural_key='C001'`.

🔗 ANALOGY

A surrogate key pipeline is like **a post office's address forwarding system**. When a person moves (source key changes), the post office updates the forwarding record (key pipeline).

Mail addressed to the old address (natural key) is delivered to the new address (same surrogate). The letters (fact rows) never need to be reprinted with a new address.

The forwarding table absorbs the change.

⚠️ WARNING

Never expose surrogate keys to end users or source systems. Surrogate keys are internal warehouse identifiers only. External-facing APIs, reports delivered to business users, and source system interfaces should always use natural keys.

A surrogate key that leaks outside the warehouse and gets stored in another system defeats its entire purpose - it becomes a source-coupled natural key.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Starting the sequence at 1 and using the same sequence for all dimension tables. `customer_key = 1` and `product_key = 1` are both valid and refer to completely different entities.

Developers write JOIN conditions that accidentally cross-join on equal integer values.

Correct Way: Use separate sequences per dimension table (each starts at 1). Or use range-allocated sequences (customers: 1–10M, products: 10M+–20M). In either case, the FK column name (`customer_key` vs `product_key`) prevents accidental cross-joins.

EXAM TIP CARD

Interview classic: "Why do dimensional warehouses use surrogate keys instead of natural keys from source systems?" Winning answer: "Three reasons. First, source key instability: source systems change, merge, or renumber.

Surrogate keys are permanent warehouse identifiers - fact rows are never touched when source keys change. Second, multi-source integration: the same entity (a product) may

have different natural keys in different source systems; one surrogate key represents all of them.

Third, SCD support: each Type 2 version of a dimension row needs its own surrogate key - natural keys cannot uniquely identify historical versions."*

🔥 EXERCISE

A retail warehouse receives product data from three source systems: (a) POS system uses `product_code VARCHAR(10)` . (b) ERP uses `item_number INT` .

(c) E-commerce platform uses `sku UUID` . Design a key pipeline table that maps all three to a single `product_key INT` surrogate.

Write the ETL lookup query that returns the correct surrogate key given any of the three natural key formats.

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"Design a complete surrogate key management system for a retail data warehouse that receives product data from three source systems (POS, ERP, e-commerce). Include: the key pipeline DDL, the ETL lookup logic (with new-key generation on miss), the handling of natural key changes without touching fact tables, and a query to detect orphaned surrogate keys (surrogates that appear in a fact table but not in the current dimension)."*

LATE-ARRIVING DIMENSIONS — WHEN FACTS ARRIVE BEFORE DIMENSIONS

Claude • AI Assistant

Late-Arriving Dimensions – Placeholder Pattern and Back-Update

Phase 1: T=0: Fact arrives before dimension

fact record

```
fact_sales:
date=today
store_id=NEW_STORE_42
product_key=8814
revenue=45000
```

dim_store lookup

```
store_id=NEW_STORE_42
→ NOT FOUND in dim_store
```

Option A: Drop fact row.
DATA LOST.
500 transactions silently gone.

Option B: Create placeholder
dim_store row:
store_key=9042,
store_code=NEW_STORE_42,
store_name=UNKNOWN-PENDING,
city=Unknown, is_placeholder=1.

Load fact with store_key=9042.
FACT PRESERVED.

Phase 2: T+4 hours: Real dimension data arrives

source record

```
store_id=NEW_STORE_42
store_name=Indiranagar Express
city=Bengaluru
store_format=Express
```

UPDATE

dim_store

```
UPDATE dim_store
SET store_name=Indiranagar Express,
    city=Bengaluru,
    is_placeholder=0
WHERE store_key=9042
```

fact_sales rows with **store_key=9042** now correctly show Bengaluru / Indiranagar Express — **NO** fact row updates needed.

Monitoring query:

```
SELECT * FROM dim_store WHERE is_placeholder=1
AND CURRENT_DATE - row_start_date > 1
```

Placeholder preserves the fact. Back-update heals the dimension. No fact rows ever need to be touched.

Late-Arriving Dimensions - Placeholder Pattern and Back-Update

sql – Late-arriving dimension – placeholder creation, fact load, and back-update

```
-- =====
-- STEP 1: ETL lookup – check if store exists
-- =====

SELECT store_key
FROM   dim_store
WHERE  store_code = 'NEW_STORE_42'
      AND is_current = 1;
-- Returns 0 rows → store dimension not yet loaded

-- =====
-- STEP 2: Create placeholder dimension row
-- =====

INSERT INTO dim_store (
    store_key, store_code, store_name, city, state, region,
    store_format, open_date, row_start_date, row_end_date,
    is_current, is_placeholder
) VALUES (
    NEXTVAL('seq_store_key'), -- real surrogate key
    'NEW_STORE_42',           -- natural key (known)
    'UNKNOWN - PENDING',      -- descriptive attrs set to Unknown
    'Unknown', 'Unknown', 'Unknown', 'Unknown',
    NULL,
    CURRENT_DATE, '9999-12-31', 1,
    1                        -- is_placeholder = 1 flag
);
```

```
-- Get the generated key
-- (in ETL: RETURNING store_key or @@IDENTITY etc.)

-- =====
-- STEP 3: Load fact rows with placeholder key
-- =====
INSERT INTO fact_sales_transaction
    (date_key, store_key, product_key, customer_key,
     transaction_number, quantity_sold, net_revenue)
SELECT
    d.date_key,
    s.store_key,          -- placeholder surrogate key
    p.product_key,
    c.customer_key,
    src.txn_number,
    src.quantity,
    src.revenue
FROM   stg_sales_source src
JOIN   dim_store          s ON s.store_code = src.store_code
      AND s.is_current = 1
JOIN   dim_date           d ON d.calendar_date = src.sale_date
JOIN   dim_product        p ON p.product_code = src.product_code
      AND p.is_current = 1
JOIN   dim_customer       c ON c.customer_id = src.customer_id
      AND c.is_current = 1;

-- =====
-- STEP 4: Back-update when real store data arrives
-- =====
UPDATE dim_store
SET
    store_name      = 'Indiranagar Express',
    city            = 'Bengaluru',
    state           = 'Karnataka',
    region          = 'South',
    store_format    = 'Express',
    open_date       = '2024-11-05',
    is_placeholder  = 0
WHERE store_code   = 'NEW_STORE_42'
    AND is_placeholder = 1;
-- No fact rows updated – they already reference the correct surrogate key

-- =====
-- MONITORING: placeholders older than 24 hours
-- =====
SELECT
```

```

    s.store_code,
    s.row_start_date                                AS
placeholder_created,
    CURRENT_DATE - s.row_start_date                AS age_days,
    COUNT(f.store_key)                             AS
affected_fact_rows
FROM    dim_store                                  s
LEFT JOIN fact_sales_transaction f ON f.store_key = s.store_key
WHERE   s.is_placeholder = 1
GROUP  BY s.store_code, s.row_start_date
HAVING CURRENT_DATE - s.row_start_date >= 1
ORDER  BY age_days DESC;

```

Student

What happens when a fact record arrives in the warehouse but the corresponding dimension row doesn't exist yet?

Claude • AI Assistant

Late-arriving dimensions are more common than most textbooks acknowledge. A sale is logged at the POS immediately, but the new store that opened today may not have been provisioned in the warehouse's `dim_store` yet.

A customer's first purchase arrives before their loyalty card registration completes in the CRM. The warehouse must handle this without losing the fact or corrupting referential integrity.

CONCEPT CARD

Why dimensions arrive late:

1. **System latency** - the source system that feeds the dimension (CRM, MDM, HR) runs on a different schedule from the source system that feeds the fact (POS, transaction processing).
2. **Data entry delay** - a new employee starts work before HR completes their profile. Their sales transactions arrive before their `dim_employee` row exists.
3. **Integration lag** - a new supplier begins shipping before their supplier master record is loaded into the warehouse.
4. **Event ordering** - in distributed systems, the fact event and the dimension creation event are generated by different services and may arrive out of order in the ETL pipeline.

CONCEPT CARD

The inferred/placeholder member pattern:

1. **When a fact arrives with an unknown dimension key** - create a placeholder dimension row immediately, with the natural key filled in and all descriptive attributes set to 'Unknown' or their domain-specific equivalent.
2. **Placeholder DDL** - `INSERT INTO dim_employee (employee_key, employee_id, employee_name, department, row_start_date, is_current) VALUES (NEXTVAL('seq_employee_key'), 'E9042', 'UNKNOWN - PENDING', 'Unknown', CURRENT_DATE, 1) .`
3. **Load the fact** - the fact row gets the placeholder `employee_key` . Referential integrity is maintained. The fact is not lost.
4. **Back-update when dimension arrives** - when the real employee record arrives in the next load: UPDATE the placeholder row in-place (Type 1 - no new surrogate needed) with the actual descriptive attributes.

All fact rows that reference this key automatically 'see' the real attributes.

5. **Tracking placeholders** - maintain a flag or separate table of `is_placeholder TINYINT = 1` rows. Run a daily job to check if real data has arrived for any placeholder.

CONCEPT CARD

Back-update mechanics:

1. **Find the placeholder** - `SELECT employee_key FROM dim_employee WHERE employee_id = 'E9042' AND is_placeholder = 1 .`
2. **Update in-place** - since the surrogate key hasn't been used in a context where its previous 'Unknown' value is analytically meaningful, Type 1 overwrite is appropriate. Replace 'Unknown' with real attributes.
3. **Clear the placeholder flag** - `UPDATE dim_employee SET employee_name = 'Ananya Krishnan', department = 'Sales', is_placeholder = 0 WHERE employee_key = 9042 .`
4. **No fact rows updated** - all fact rows continue to reference `employee_key = 9042` . Now that the dimension row has real data, those fact rows automatically report correctly.

🔗 ANALOGY

The placeholder member is like **reserving a seat on a flight for a passenger whose name isn't confirmed yet**. The seat (fact row) is booked.

The reservation (placeholder dimension row) holds the slot. When the passenger's name is confirmed (real dimension data arrives), you update the name on the reservation.

The seat assignment doesn't change - the passenger is now correctly identified in the system.

⚠️ WARNING

Do not use NULL foreign keys as a substitute for placeholder members. A NULL `employee_key` in the fact table breaks GROUP BY aggregations, breaks referential integrity constraints, and prevents any dimension-attribute filtering of that fact row. Always use a well-known surrogate (like -1 or a generated placeholder key) rather than NULL.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Holding the fact row in a staging area indefinitely, waiting for the dimension to arrive. If the dimension never arrives (data entry error), the fact is permanently stuck in staging - not in the warehouse. The load appears successful but data is quietly lost.

Correct Way: Create a placeholder immediately and load the fact. Track all placeholders. Alert the data engineering team when a placeholder is more than N hours old without resolution. Data is in the warehouse; incompleteness is visible and monitored.

EXAM TIP CARD

Interview classic: "A fact record arrives but the customer dimension row for that customer doesn't exist yet. What do you do?" Winning answer: "Create an inferred/placeholder member."

Insert a dimension row with the customer's natural key and all descriptive attributes set to 'Unknown'. Assign a regular surrogate key.

Load the fact row using this surrogate. Flag the placeholder row with `is_placeholder=1`.

When the real customer record arrives in a later load, back-update the placeholder row in-place (Type 1 update) with real attributes. All fact rows referencing this surrogate automatically become correct.

Never use NULL foreign keys or hold facts in staging indefinitely."*

🔥 EXERCISE

Your `fact_sales_transaction` table requires a valid `store_key` foreign key. A new store opened today and sold 500 transactions before its `dim_store` row was loaded.

Write: (a) the placeholder INSERT for the new store, (b) the fact load using the placeholder key, (c) the back-update when the real store record arrives, (d) the monitoring query that shows all placeholders older than 24 hours.

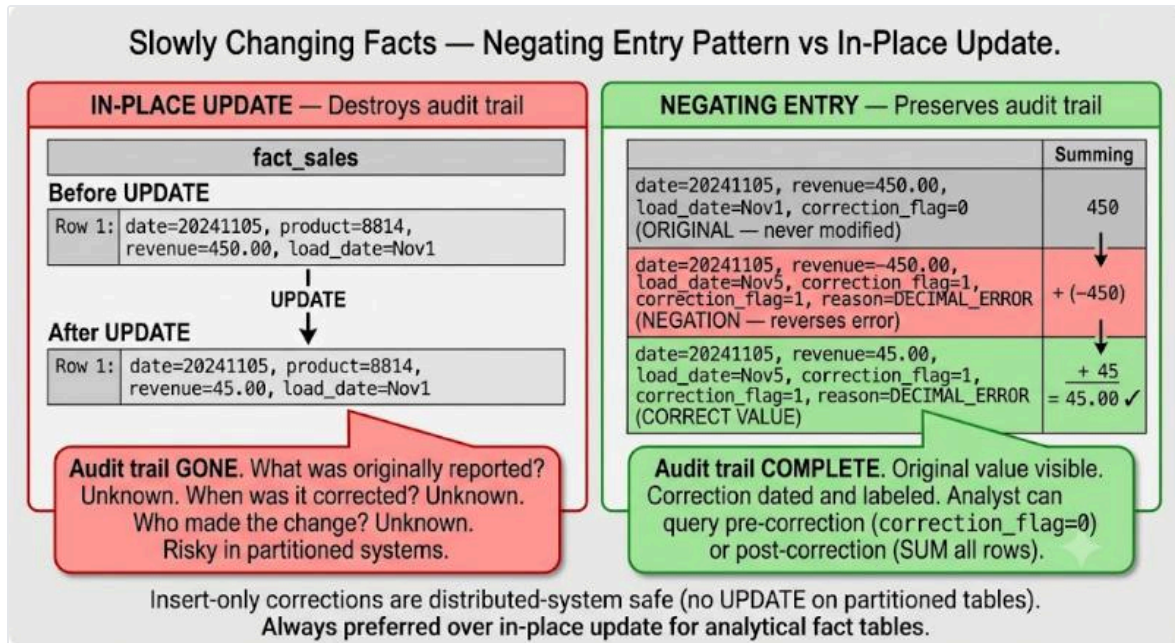
🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a complete late-arriving dimension handling system for a retail warehouse. Include: the `is_placeholder` flag in the dimension DDL, the ETL logic for creating placeholders vs back-updating them, a monitoring dashboard query showing

placeholder age and affected fact row count, and an alerting threshold (when to escalate a stale placeholder to the data engineering team)."

SLOWLY CHANGING FACTS — CORRECTIONS AND RESTATEMENTS

Claude • AI Assistant



Slowly Changing Facts - Negating Entry Pattern vs In-Place Update

sql – Negating entry pattern – duplicate batch correction and verification

```
-- =====
-- ADD CORRECTION METADATA COLUMNS TO FACT TABLE
-- (add at table creation time – not as an afterthought)
-- =====
ALTER TABLE fact_sales_transaction ADD COLUMN
    load_date          DATE          NOT NULL DEFAULT CURRENT_DATE;
ALTER TABLE fact_sales_transaction ADD COLUMN
    correction_flag     TINYINT       NOT NULL DEFAULT 0;    -- 0=original,
1=correction
ALTER TABLE fact_sales_transaction ADD COLUMN
    correction_reason   VARCHAR(50);    -- NULL for originals

-- =====
-- SCENARIO: ETL job ran twice on Nov 5 – 50k rows duplicated
-- STEP 1: Detect the problem
```

```

-- =====
SELECT
    load_date,
    COUNT(*)          AS row_count,
    SUM(net_revenue) AS total_revenue
FROM    fact_sales_transaction
WHERE   correction_flag = 0          -- original rows only
    AND date_key = 20241105
GROUP BY load_date
ORDER BY load_date;
-- If load_date shows TWO rows for Nov 5 → duplicate batch confirmed

-- =====
-- STEP 2: Negating INSERT – reverses all duplicate rows
-- (insert one negation row per original duplicate)
-- =====
INSERT INTO fact_sales_transaction
    (date_key, store_key, product_key, customer_key, transaction_number,
     quantity_sold, net_revenue, tax_amount,
     load_date, correction_flag, correction_reason)
SELECT
    date_key, store_key, product_key, customer_key, transaction_number,
    -quantity_sold,      -- NEGATED
    -net_revenue,        -- NEGATED
    -tax_amount,         -- NEGATED
    CURRENT_DATE,
    1,                  -- correction_flag
    'DUPLICATE_BATCH_NOV5'
FROM    fact_sales_transaction
WHERE   load_date      = '2024-11-05'  -- the duplicate load date
    AND correction_flag = 0;           -- original rows only – not already-
corrected

-- =====
-- STEP 3: Verify – net revenue should equal the correct total
-- =====
SELECT
    SUM(CASE WHEN correction_flag = 0 THEN net_revenue ELSE 0 END) AS
before_correction,
    SUM(net_revenue) AS
after_correction,
    COUNT(*)
    FILTER (WHERE correction_flag = 1) AS
correction_rows
FROM    fact_sales_transaction
WHERE   date_key = 20241105;

```

```
-- before_correction = 2x correct value (duplicate)
-- after_correction  = 1x correct value (net of negation)

-- =====
-- AUDIT REPORT: all corrections applied to a date
-- =====

SELECT
    load_date,
    correction_reason,
    COUNT(*)          AS correction_rows,
    SUM(net_revenue) AS net_correction_amount
FROM fact_sales_transaction
WHERE correction_flag = 1
    AND date_key       = 20241105
GROUP BY load_date, correction_reason;
```

Student

What happens when a measure in an already-loaded fact row turns out to be wrong?
How do I correct it without destroying the audit trail?

Claude • AI Assistant

Facts are supposed to be immutable - a sale happened at a point in time and the record should reflect that. But source systems make errors: a decimal point in the wrong place, a batch that processed twice, a return that was misclassified.

The warehouse must handle corrections while preserving the audit trail that tells analysts what was reported at any point in time.

CONCEPT CARD

Three approaches to fact correction:

1. **In-place update** - overwrite the wrong value with the correct value. Simple. Destroys the audit trail. Acceptable only for obvious data entry errors with no analytical impact (wrong decimal in a test row).
2. **Negating entry (reversal)** - insert a new row with the same keys but negated measure values, then insert a third row with the correct values. Net effect: the original error + the negation + the correction = correct total. Preserves full audit trail.
3. **Versioned correction table** - maintain a separate `fact_sales_correction` table. The correction rows reference the original fact via a degenerate `correction_reference` key.

Query both tables to get the net correct total.

CONCEPT CARD

The negating entry pattern - the production standard:

1. **Original row (Day 1)** - `(date_key=20241105, product_key=8814, revenue=450.00)` - loaded correctly.
2. **Error discovered (Day 5)** - revenue was actually 45.00 (decimal error).
3. **Negating row (Day 5)** - `INSERT (date_key=20241105, product_key=8814, revenue=-450.00, correction_flag=1, original_load_date=20241105)` - reverses the error.
4. **Correct row (Day 5)** - `INSERT (date_key=20241105, product_key=8814, revenue=45.00, correction_flag=1)` - correct value.
5. **Net at query time** - `SUM(revenue) = 450 + (-450) + 45 = 45.00` . Correct total. Audit trail: three rows visible, correction lineage clear.

CONCEPT CARD

Correction metadata columns:

1. **'load_date DATE'** - the ETL load date for this row. Distinguishes original load from correction load.
2. **'correction_flag TINYINT DEFAULT 0'** - 1 for negation or correction rows; 0 for original rows.
3. **'correction_reason VARCHAR'** - 'DECIMALERROR', 'DUPLICATEBATCH', 'MISCLASSIFICATION'. Documents why the correction was made.
4. **'corrected_by VARCHAR'** - the ETL job or analyst who loaded the correction.
5. **Reporting filter** - standard reports use `SUM(revenue)` across all rows including corrections (netting to correct).

Audit reports use `WHERE correction_flag = 0` to see pre-correction picture. Correction reports use `WHERE correction_flag = 1` to see all corrections applied.

EXAMPLE CARD

Duplicate batch correction:

1. An ETL job ran twice. 50,000 rows were loaded twice. All amounts are doubled.
2. **Detection** - `SELECT date_key, COUNT(*), SUM(revenue) FROM fact_sales WHERE load_date = '2024-11-05' GROUP BY date_key` shows double the expected row count.
3. **Fix: negating entry** - for each duplicated row, INSERT a negation row.

```
INSERT INTO fact_sales SELECT date_key, store_key, product_key, customer_key,
transaction_number, -quantity_sold, -net_revenue, CURRENT_DATE, 1,
'DUPLICATE_BATCH' FROM fact_sales WHERE load_date = '2024-11-05' .
```

4. **Net effect** - every row now has an equal and opposite partner. `SUM(revenue)` for Nov 5 → original correct total.
5. **The original rows are NOT deleted** - the audit trail is preserved. Anyone can query `WHERE correction_flag = 0` to see exactly what was first loaded.

🔗 ANALOGY

Negating entries are like **double-entry bookkeeping in accounting**. A wrong debit entry is never erased - it is reversed with a credit entry, then a correct debit entry is recorded.

The ledger shows all three entries. The net balance is correct.

The audit trail is complete. Auditors can see every correction and why it was made.

⚠️ WARNING

In-place UPDATE of fact rows is dangerous in partitioned tables. In systems like BigQuery, Redshift, or Snowflake, UPDATE operations on very large partitioned tables are expensive and can corrupt partition statistics. The negating entry pattern requires only INSERTs - much safer for distributed columnar systems.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: `DELETE FROM fact_sales WHERE load_date = '2024-11-05'` followed by a reload. The delete destroys the audit trail.

If the re-load also has a bug, there is no way to reconstruct what the data looked like before the correction cycle. Also, other downstream systems may have already consumed the deleted rows.

Correct Way: Negating entries. Original rows stay. Corrections are inserted with `correction_flag = 1`. Net totals are correct. Audit trail is complete. Downstream systems see corrections as net-zero adjustments.

EXAM TIP CARD

Interview classic: "How do you correct a fact table row that was loaded with a wrong amount?" Winning answer: "Negating entry pattern. Don't update or delete the original row - insert a negation row with the same dimension keys and negated measures, plus correction metadata (`correctionflag=1`, `correctionreason`, `load_date`).

Then insert a third row with the correct values. Sum across all three rows nets to the correct total.

The audit trail is preserved: original load, reversal, and correction are all visible and dated. This approach is insert-only, compatible with distributed systems, and preserves full lineage."

🔥 EXERCISE

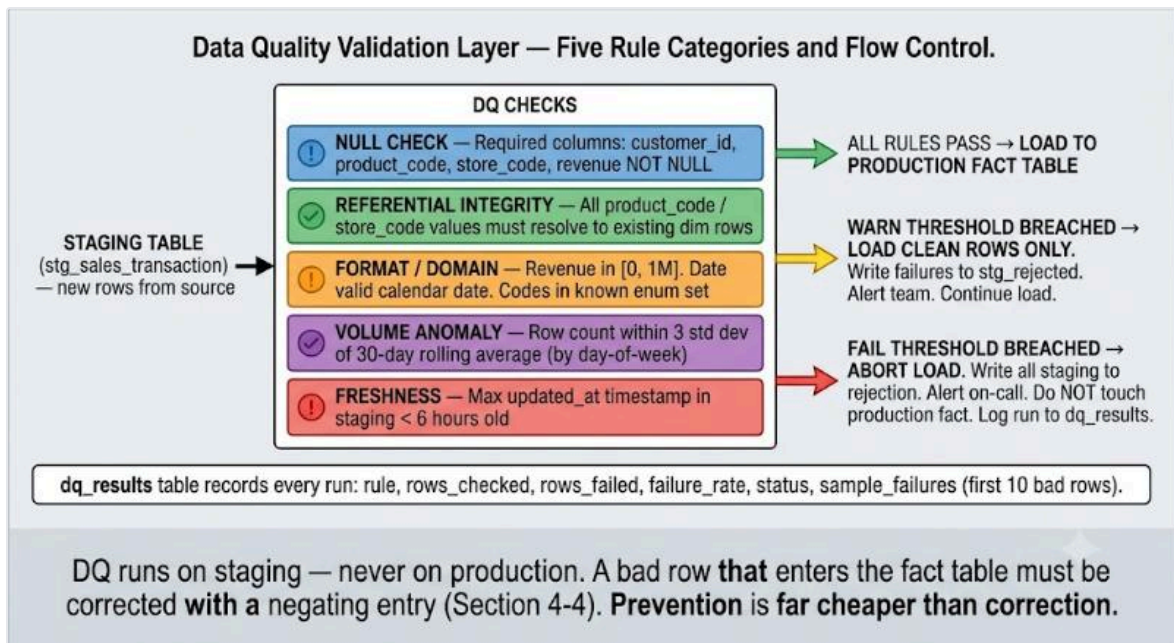
A fact table has 1,000 rows for November 5 where **tax_amount** was computed as 18% instead of the correct 12%. Write: (a) the negating INSERT that reverses all wrong tax amounts, (b) the correcting INSERT with the correct 12% tax amount, (c) a verification query that shows the net tax_amount after corrections equals the 12% total.

🔮 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a complete fact correction framework for a financial data warehouse. Include: the correction metadata columns added to fact tables, the negating entry INSERT pattern for three types of errors (wrong amount, duplicate batch, wrong dimension key), a correction audit report query, and a data lineage query that shows the full correction history for any given fact row."

DATA QUALITY RULES — THE VALIDATION LAYER

Claude • AI Assistant



Data Quality Validation Layer - Five Rule Categories and Flow Control

sql — Five DQ rule implementations for a sales staging table

```
-- =====
-- RULE 1: NULL CHECK on required columns
-- =====

SELECT
```

```

'null_check' AS rule_name,
COUNT(*) AS rows_checked,
SUM(CASE WHEN customer_id IS NULL
          OR product_code IS NULL
          OR store_code IS NULL
          OR net_revenue IS NULL THEN 1 ELSE 0 END) AS rows_failed
FROM stg_sales_transaction;

-- =====
-- RULE 2: REFERENTIAL INTEGRITY – product_code resolves to dim
-- =====
SELECT
  'ref_integrity_product' AS rule_name,
  COUNT(*) AS rows_checked,
  SUM(CASE WHEN p.product_key IS NULL THEN 1 ELSE 0 END) AS rows_failed
FROM stg_sales_transaction src
LEFT JOIN dim_product p
      ON p.product_code = src.product_code
      AND p.is_current = 1;

-- Route orphan rows to rejection table
INSERT INTO stg_sales_rejected
SELECT src.*, CURRENT_DATE, 'UNRESOLVABLE_PRODUCT_CODE'
FROM stg_sales_transaction src
LEFT JOIN dim_product p
      ON p.product_code = src.product_code AND p.is_current = 1
WHERE p.product_key IS NULL;

-- =====
-- RULE 3: RANGE / DOMAIN CHECK
-- =====
SELECT
  'range_check_revenue' AS rule_name,
  COUNT(*) AS rows_checked,
  SUM(CASE WHEN net_revenue < 0
          OR net_revenue > 1000000 THEN 1 ELSE 0 END) AS rows_failed,
  MIN(net_revenue) AS min_val,
  MAX(net_revenue) AS max_val
FROM stg_sales_transaction;

-- =====
-- RULE 4: VOLUME ANOMALY (Z-score against 30-day rolling avg)
-- =====
WITH daily_counts AS (
  SELECT
    load_date,

```

```

        DAYOFWEEK(load_date)          AS dow,
        COUNT(*)                      AS row_count
    FROM    fact_sales_transaction
    WHERE   load_date >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY load_date
),
dow_stats AS (
    SELECT
        dow,
        AVG(row_count)    AS avg_count,
        STDDEV(row_count) AS std_count
    FROM daily_counts
    GROUP BY dow
)
SELECT
    (SELECT COUNT(*) FROM stg_sales_transaction) AS today_count,
    ds.avg_count,
    ds.std_count,
    ABS((SELECT COUNT(*) FROM stg_sales_transaction) - ds.avg_count)
    / NULLIF(ds.std_count, 0)          AS z_score,
    CASE WHEN ABS((SELECT COUNT(*) FROM stg_sales_transaction) -
ds.avg_count)
        / NULLIF(ds.std_count, 0) > 3
        THEN 'ANOMALY - INVESTIGATE'
        ELSE 'NORMAL'
    END                                AS volume_status
FROM dow_stats ds
WHERE ds.dow = DAYOFWEEK(CURRENT_DATE);

-- =====
-- RULE 5: FRESHNESS CHECK
-- =====
SELECT
    MAX(source_updated_at)          AS latest_source_ts,
    CURRENT_TIMESTAMP - MAX(source_updated_at)    AS lag_interval,
    CASE WHEN CURRENT_TIMESTAMP - MAX(source_updated_at) > INTERVAL '6 hours'
        THEN 'STALE - PIPELINE MAY BE BROKEN'
        ELSE 'FRESH'
    END                                AS freshness_status
FROM stg_sales_transaction;

```

What data quality checks should run before every load, and how do I structure them so a failure stops the pipeline?

Claude • AI Assistant

Data quality validation is the gate between staging and production. A bad row that reaches a fact table corrupts every aggregation that touches it - and may not be discovered for weeks.

A well-designed DQ layer catches bad rows in staging, routes them to an error table, alerts the team, and prevents partial loads from creating inconsistent warehouse states.

CONCEPT CARD

Five core DQ rule categories:

1. **Null checks** - required columns must not be NULL. Dimension natural keys, fact measure columns marked NOT NULL in the DDL must be validated in staging before load.
2. **Referential integrity** - every FK value in the staging fact table must resolve to an existing row in the corresponding dimension. 'Does this *productcode* exist in *dimproduct*?'
3. **Format and domain checks** - email addresses contain '@', dates are valid calendar dates, codes match known enumeration values, measures are within plausible ranges.
4. **Volume and row-count anomaly** - today's staging load has 50,000 rows. The last 30 days averaged 48,000 rows per day. Acceptable. But if today shows 500,000 or 5,000 rows - anomaly, investigate before loading.
5. **Freshness checks** - the last record in staging has `updated_at` of 3 days ago. The source system hasn't sent data in 3 days. Pipeline may be broken. Alert.

CONCEPT CARD

DQ result table - structure:

1. ``dq_results(rule_name, table_name, run_date, rows_checked, rows_failed, failure_rate, status, sample_failures)``.
2. ``status`` - 'PASS', 'WARN', 'FAIL'. FAIL aborts the load. WARN loads with a flag. PASS proceeds normally.
3. ``sample_failures`` - store the first 10 failing rows for debugging.
4. **Threshold configuration** - store acceptable failure rates per rule:
`dq_thresholds(rule_name, warn_pct, fail_pct)` . 'null_check: WARN at 0.1%, FAIL at 1.0%.' Configurable without code changes.

CONCEPT CARD

Referential integrity check - the most critical fact DQ rule:

1. **Check before loading any fact row** - `LEFT JOIN stg_sales TO dim_product. WHERE dim match IS NULL → unresolvable FK.`
2. **Two responses to orphan FKs:**
 - If `< fail_threshold` → route orphan rows to `stg_sales_rejected` . Load only the clean rows. Log the orphans.
 - If `>= fail_threshold` → abort entire load. Something systemic is broken.
5. **Orphan rows in staging** - stored in a rejection table with the natural key that couldn't be resolved: `stg_sales_rejected(all original columns, reject_reason, reject_date)` .
6. **Reprocessing** - after the dimension issue is fixed, reprocess from the rejection table.

CONCEPT CARD

Volume anomaly detection:

1. **Baseline** - compute rolling 30-day average and standard deviation of daily row count from recent loads.
2. **Alert condition** - today's count is more than 3 standard deviations from the mean (Z-score > 3 or < -3).
3. **Zero-row condition** - if the staging table has 0 rows, always FAIL - a source system failure is far more likely than a genuinely empty day.
4. **Day-of-week seasonality** - Sundays may legitimately have 40% fewer rows than Mondays. Build day-of-week baselines separately: compare today vs the same day of the week average.

🔗 ANALOGY

The DQ validation layer is like **quality control inspection on a factory production line**. Raw materials (staging data) enter the inspection station before reaching the production floor (warehouse).

Defective units (bad rows) are routed to a reject bin (rejection table) rather than mixed into the finished product. The line stops (load aborted) if the defect rate exceeds the acceptable threshold.

No defective product ships.

⚠️ WARNING

Silent row drops are the worst DQ outcome. If a fact row with an unresolvable FK is simply skipped without logging, it disappears without a trace. Analysts notice a discrepancy between source system counts and warehouse counts - but cannot explain or reproduce it.

Every rejected or skipped row must be written to a rejection table with the rejection reason. Silence is not acceptable.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Running DQ checks AFTER loading into the production fact table. Now the bad rows are mixed with good rows. Rolling back a fact load is expensive and sometimes impossible in append-only systems.

Correct Way: Run ALL DQ checks on staging data BEFORE loading into production. Only clean rows cross into the warehouse. Bad rows go to rejection tables. The warehouse never sees a bad row.

EXAM TIP CARD

Interview classic: "What data quality checks do you run before loading a fact table?" Winning answer: *"Five categories: (1) Null checks - required columns (natural keys, not-null measures) must not be NULL. (2) Referential integrity - every FK in the staging fact must resolve to an existing dimension row.

(3) Format/domain checks - valid dates, correct code values, measures within plausible range. (4) Volume anomaly - today's row count within 3 standard deviations of the 30-day rolling average (with day-of-week seasonality).

(5) Freshness - source data timestamp is recent enough to indicate the pipeline is healthy. All checks run on staging; only clean rows reach the warehouse."*

🔥 EXERCISE

Write DQ check queries for a `stg_sales_transaction` staging table: (a) null check on `customer_id`, `product_code`, `store_code`, `revenue`, (b) referential integrity check - find source rows where `product_code` has no matching `dim_product` row, (c) range check - flag rows where `revenue < 0 OR revenue > 1000000`, (d) volume check - compare today's row count to the 30-day average.

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Build a complete data quality framework for a dimensional warehouse ETL pipeline. Include: a `dq_results` table DDL, a `dq_thresholds` configuration table, SQL implementations of all five DQ rule categories (null, referential integrity, format, volume anomaly, freshness), a rejection table DDL, and the ETL control logic that reads threshold configuration, runs checks, and either proceeds, warns, or aborts based on failure rates."

CHAPTER REVIEW & SUMMARY

CHAPTER SUMMARY

You now have the implementation toolkit for building and loading a production dimensional warehouse - the SQL mechanics for every ETL phase from staging through validation through load.

- **SCD TYPE 2 MERGE:** Required dimension columns: natural key, surrogate key, tracked attributes, *rowstartdate*, *rowenddate* (convention: 9999-12-31 = active), *iscurrent*. *Three MERGE outcomes: NOT MATCHED → INSERT new (generate surrogate, start=today, end=9999-12-31, iscurrent=1); MATCHED AND changed → UPDATE (expire: end=today-1, iscurrent=0) then INSERT new version; MATCHED AND unchanged → no action. Two-step pattern: Step 1 MERGE to expire, Step 2 INSERT new versions. Consistency check: every iscurrent=1 row must have row_end='9999-12-31' and vice versa.*
- **SURROGATE KEY GENERATION:** Three mechanisms - database sequence/identity (standard), key pipeline table (multi-source, natural key change handling), hash surrogate (distributed pipelines). Key pipeline maintains *naturalkey* → *surrogatekey* mapping per *source_system*. Absorbs source key changes without touching fact rows. Each Type 2 version gets its own surrogate key. Surrogate keys are internal warehouse identifiers only - never expose to source systems or end users.
- **LATE-ARRIVING DIMENSIONS:** When a fact arrives with no matching dimension row - create a placeholder/inferred member immediately. Placeholder has natural key filled in; all descriptive attributes set to 'Unknown'; *is_placeholder=1* flag. Load fact with placeholder surrogate key. When real dimension data arrives, back-update the placeholder in-place (Type 1). All referencing fact rows automatically correct. Monitor placeholder age; alert when stale. Never use NULL FKs; never hold facts in staging indefinitely.
- **SLOWLY CHANGING FACTS:** Three correction approaches - in-place UPDATE (destroys audit trail, use only for obvious test errors), negating entry (production standard - insert negation row + correction row; SUM nets to correct value; audit trail preserved), versioned correction table (separate correction fact). Negating entry is insert-only, safe for distributed systems. Add correction metadata: *loaddate*, *correctionflag*, *correctionreason*. *Standard reports SUM all rows. Audit reports filter correctionflag=0* for pre-correction view.
- **DATA QUALITY RULES:** Five categories - null checks (required columns), referential integrity (FKs resolve to dimensions), format/domain checks (ranges, enumerations, dates), volume anomaly (Z-score vs 30-day rolling average, day-of-week adjusted), freshness (source data lag < threshold). Run on staging, never on production. Results to *dq_results* table. Rejected rows to rejection table with reason. FAIL threshold aborts load; WARN loads clean rows only. Silence on bad rows is never acceptable.

 Claude • AI Assistant

The implementation layer is complete. Chapter 5 synthesizes everything into the end-to-end design workflow - from a business requirement statement through a Bus Matrix through a physical model - then shows the analytics that dimensional models enable (revenue trends, RFM segmentation, LTV), catalogs the most common modeling mistakes, and closes with the mock design interview that prepares you to whiteboard a dimensional model aloud under interview conditions.